

ST. FRANCIS INSTITUTE OF TECHNOLOGY
DEPARTMENT OF INFORMATION TECHNOLOGY
Computer Network Security LAB

Experiment – 5: Implementation of Digital Signature Scheme

Aim: Write a program to implement RSA Digital Signature Scheme.

Objective: After performing the experiment, the students will be able to –

- To understand the RSA Digital Signature Scheme

Lab objective mapped:

L502.3: Students should be able to Analyze and evaluate performance of digital signature scheme and demonstrate key management.

Prerequisite: Basic knowledge of Digital Signature.

Requirements: PYTHON

Pre-Experiment Theory:

Digital Signature Requirements:

- The signature must be a bit pattern that depends on the message being signed.
- The signature must use some information unique to the sender to prevent both forgery and denial.
- It must be relatively easy to produce the digital signature.
- It must be relatively easy to recognize and verify the digital signature.
- It must be computationally infeasible to forge a digital signature, either by constructing a new message for an existing digital signature or by constructing a fraudulent digital signature for a given message.

RSA can be used for signing and verifying a message. In this case it is called the RSA digital signature scheme.

The digital signature scheme changes the roles of the private and public keys.

1. First, the private and public key of the sender, not the receiver are used.
2. Second, the sender uses his/her own private key to sign the document; the receiver uses the sender's public key to verify it.

Generation of RSA Key Pair

- Key generation in the RSA digital signature scheme is exactly the same as key generation in the RSA cryptosystem.
- Alice chooses two primes p and q and calculates $n = p \times q$.
- Alice calculates $\phi(n) = (p - 1)(q - 1)$.
- She then chooses e , the public exponent, and calculates d , the private exponent such that $e \times d = 1 \bmod \phi(n)$.
- Alice keeps d , she publicly announces n and e .

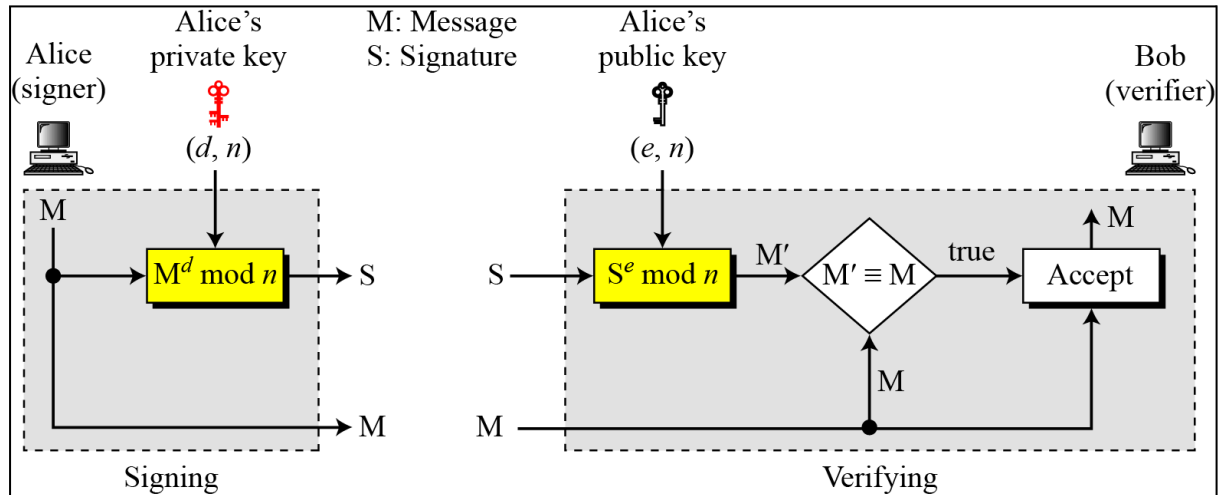
Signing

Alice creates a signature out of the message using her private exponent, $S = M^d \bmod n$ and sends the message and the signature to Bob.

Verifying

Bob receives M and S. Bob applies Alice's public exponent to the signature to create a copy of the message $M' = S^e \bmod n$.

Bob compares the value of M' with the value of M. If the two values are congruent, Bob accepts the message.



Procedure:

Write a program in Python for key generation, signing and verification using the RSA algorithm.

- For Key generation, ask the user to enter the value of prime numbers p & q and a public key 'e'. (Note that values of p , q & e cannot be random, they should satisfy criteria as per RSA algorithm)
 - Program should calculate the private key element 'd' using Extended Euclidean Algorithm.
 - Provide a set of public (e, n) and private key (d, n) as the output to the user.
 - For digital sign generation, ask the user to enter the Message. Using a private key, calculate the digital sign 'S' and output the same.
 - For sign verification, ask the user to enter the sign 'S' and the Message 'M'. Using the public key, calculate the M' value. Compare this M' value with Message 'M'. If they are the same, display that "The message is authenticate" else display "Message is altered, Discard."
2. Test the output of program for following exercises:
- For $p=11$, $q=3$ and $e=3$, find public and private keys using the RSA algorithm. For message, $M=10$, Find the digital signature value. Also show how any receiver of this signature will verify it.
 - For $p=7$, $q=11$, $e=13$. Find public and private keys using the RSA algorithm. For message, $M=5$, Find the digital signature value. Also show how any receiver of this signature will verify it.

Output:

- Attach the complete code performing key generation, signature generation and verification.
- Attach the program output for key generation (display public key & private key), digital sign 'S' and verification code M' for the inputs given in the exercise above.

Post Experimental Exercise:

1. Solve both the exercises mentioned in the procedure on journal sheets. [Theoretical result and attached code's output should match].
2. What are the different attacks possible on RSA digital signature scheme?

Conclusion:

In RSA Digital Signature Scheme the signing and verifying sites use the same function but with different parameters. The verifier compares the message and the output of the function for congruence. If the result is true, the message is accepted.

CODE:

```
# RSA Digital Signature
```

```
# Function to find GCD
```

```
def gcd(a, b):
    while b != 0:
        a, b = b, a % b
    return a
```

```
# Extended Euclidean Algorithm
```

```
def extended_gcd(a, b):
    if b == 0:
        return a, 1, 0

    gcd_value, x1, y1 = extended_gcd(b, a % b)

    x = y1
    y = x1 - (a // b) * y1

    return gcd_value, x, y
```

```
# Function to find private key d
```

```
def find_d(phi, e):
    gcd_value, x, y = extended_gcd(e, phi)
    d = x % phi
    return d
```

```
# Function to check whether a number is prime
```

```
def is_prime(num):
    if num <= 1:
        return False

    for i in range(2, int(num ** 0.5) + 1):
        if num % i == 0:
            return False

    return True
```

```
# Key Generation
```

```
print("----- RSA KEY GENERATION -----")
```

```
p = int(input("Enter prime number p: "))
q = int(input("Enter prime number q: "))
e = int(input("Enter public key e: "))
```

```

# Check whether p and q are prime
if not is_prime(p) or not is_prime(q):
    print("Invalid input. p and q must be prime numbers.")

else:
    # Calculate n
    n = p * q

    # Calculate phi(n)
    phi = (p - 1) * (q - 1)

    print("n =", n)
    print("phi(n) =", phi)

    # Check whether e is valid
    if e <= 1 or e >= phi:
        print("Invalid value of e. e must be greater than 1 and less than phi(n).")

    elif gcd(e, phi) != 1:
        print("Invalid value of e. e and phi(n) must be coprime.")

    else:
        # Calculate private key d
        d = find_d(phi, e)

        print("Public Key =", (e, n))
        print("Private Key =", (d, n))

        # Digital Signature Generation
        print("\n----- DIGITAL SIGNATURE GENERATION -----")

        m = int(input("Enter Message M: "))

        if m >= n:
            print("Message must be less than n.")

        else:
            # Generate digital signature
            s = pow(m, d, n)

            print("Digital Signature S =", s)

            # Digital Signature Verification
            print("\n----- DIGITAL SIGNATURE VERIFICATION -----")

            s_received = int(input("Enter received Sign S: "))
            m_received = int(input("Enter received Message M: "))

            # Calculate M' using public key
            m_prime = pow(s_received, e, n)

            print("Calculated M' =", m_prime)

            # Compare M' with received Message M
            if m_prime == m_received:
                print("The message is authenticate")
            else:

```

```
print("Message is altered, Discard.")
```

OUTPUT:

- a. For $p=11$, $q=3$ and $e=3$, find public and private keys using the RSA algorithm. For message, $M=10$, Find the digital signature value. Also show how any receiver of this signature will verify it.

```
>>> = RESTART: C:/Users/Student/AppData/Local/Programs/Python/Python311/slexp5.py
----- RSA KEY GENERATION -----
Enter prime number p: 11
Enter prime number q: 3
Enter public key e: 3
n = 33
phi(n) = 20
Public Key = (3, 33)
Private Key = (7, 33)

----- DIGITAL SIGNATURE GENERATION -----
Enter Message M: 10
Digital Signature S = 10

----- DIGITAL SIGNATURE VERIFICATION -----
Enter received Sign S: 10
Enter received Message M: 10
Calculated M' = 10
The message is authenticate
|
```

- b. For $p=7$, $q=11$, $e=13$. Find public and private keys using the RSA algorithm. For message, $M=5$, Find the digital signature value. Also show how any receiver of this signature will verify it.

```
>>> = RESTART: C:/Users/Student/AppData/Local/Programs/Python/Python311/slexp5.py
----- RSA KEY GENERATION -----
Enter prime number p: 7
Enter prime number q: 11
Enter public key e: 13
n = 77
phi(n) = 60
Public Key = (13, 77)
Private Key = (37, 77)

----- DIGITAL SIGNATURE GENERATION -----
Enter Message M: 5
Digital Signature S = 47

----- DIGITAL SIGNATURE VERIFICATION -----
Enter received Sign S: 47
Enter received Message M: 5
Calculated M' = 5
The message is authenticate
>>> |
```